

这是一个非常经典且直击大模型核心的问题！

当输入只有“我”这一个字时，模型其实是“没有任何历史包袱”的。它生成下一个字“爱”的过程，是一次完整的前向传播(**Forward Pass**)。

我们可以把这个过程拆解为 4 个清晰的步骤：

第一步：把“我”变成数学向量 (Embedding)

大模型不认识汉字，它只认识数字。

1. 分词(**Tokenization**)：模型在它的超级字典(词表)里找到“我”的专属编号(比如 ID: 1256)。
2. 词向量(**Token Embedding**)：模型把编号 1256 映射成一个高维的浮点数向量，代表“我”的初始含义。
3. 位置编码(**Positional Encoding**)：告诉模型“这是这句话的第 1 个字”。

两者相加，我们就得到了代表“我”的初始输入矩阵 X_1 。

第二步：穿越 Transformer 神经网络 (计算与提炼)

接下来， X_1 要穿过模型内部的几十层神经网络(比如 32 层)。

因为当前只有“我”这一个字，所以在**自注意力机制(Self-Attention)**中：

- “我”算出了自己的 Q_1, K_1, V_1 。
- 因为前面没有别的字，它的 Q_1 只能和自己的 K_1 匹配，提取自己的 V_1 。(注：同时它会乖乖地把 K_1, V_1 存进刚才我们聊过的 *KV Cache* 里，为下一步做准备)。

穿过所有层之后，最初那个简单的 X_1 ，变成了一个包含了模型庞大知识库中关于“我”字深刻理解的最终隐藏状态向量(**Hidden State**)，我们叫它 H_1 。

第三步：全连接层打分 (LM Head 预测)

这是揭晓答案的关键环节。

模型要把 H_1 转化为对下一个字的预测。它会把 H_1 乘以一个巨大的词表权重矩阵(**LM Head**)。

- 假设模型的字典里有 10 万个词。
- 这个乘法做完后，会输出一个长度为 10 万的数组，里面是一堆原始的打分(叫做 **Logits**)。
- 每一个分数，对应字典里的一个词。

第四步：转化为概率并做选择 (Softmax & Sampling)

原始打分有正有负，不好比较。模型会用一个叫 **Softmax** 的数学函数，把这 10 万个打分变成 0 到 1 之间的概率值，并且所有词的概率加起来正好等于 100%。

此时，排在前面的词可能是这样的：

- “是”：15%
- “想”：12%
- “爱”：**10%**
- “去”：8%
-
- “苹果”：0.0001%

最后怎么定下“爱”字？

这就取决于你的解码策略(**Decoding Strategy**)：

- 如果你设置 $\text{Temperature} = 0$ (贪婪解码), 模型会死板地选概率最高的“是”, 生成“我是”。
- 如果你设置了稍微高一点的 Temperature 和 Top-p (比如 ChatGPT 的默认设置), 模型会掷骰子。在概率靠前的几个词里随机抽一个。这一次, 骰子正好落在了概率为 10% 的**“爱”**字上。

就这样, 在经过几百亿次矩阵乘法运算后, 屏幕上蹦出了下一个字: “爱”。